

A Beginner's Guide to the Linux Command Line

Everything you need for your first week in the terminal

A note before you begin. I put these materials together myself and revise them often, so an occasional error or typo can slip through. If something here does not behave the way I describe when you run it, please tell me.

I put this guide together from recommendations I found online and arranged it for our class. It assumes you have used a computer before but have never typed a command into a terminal, which is exactly the right place to start. Read a section, try the commands in your own terminal, and do not move on until the output makes sense. You will not break anything by looking around, and the few commands that can delete files are clearly marked.

Our environment. This guide assumes you are on the XUbuntu virtual machine, since that is our shared setup, although you may actually be on something else by the time you read this. Almost every command below is identical on Void, on Arch, and on a Mac, and when something is specific to one system, this guide says so.

1. What Linux actually is

People say Linux to mean a few different things, so it helps to separate them.

- The **kernel** is the core program that talks to the hardware. Strictly, this is what the word Linux names.
- A **distribution** bundles that kernel with a collection of tools, a package manager, and default settings. Void, Arch, and Ubuntu are three different distributions.
- The **shell** is the program that reads the commands you type and runs them. The common one, and the one you are using, is called **bash**.

You do not need to remember these distinctions perfectly. Keep the picture in mind: you type into the shell, the shell asks the kernel to do the work, and the distribution is the particular bundle you happen to be running.

2. Opening a terminal

In XUbuntu, open the **Applications** menu and choose **Terminal Emulator**. A window opens with a short line of text and a blinking cursor. That line is the **prompt**, and the cursor is where your typing appears. Nothing happens until you type a command and press **Enter**.

3. Reading the prompt

A typical prompt looks like this.

```
you@machine:~$  
# user      host      location  waiting-for-you
```

It is packed with information. Before the @ is your **user name**. After it is the **machine name**, which will read **void** or **xubuntu** depending on the computer you are using. After the colon is your **current location** in the filesystem, where ~ is shorthand for your home folder. The \$ at the end simply marks where your typing begins.

4. Where files live

Linux organizes everything into one big tree that starts at a single root, written as a plain forward slash.

- / is the **root**, the top of the whole tree.
- /home/student is your personal **home** directory, where your files go.
- ~ is a shortcut that always means your home directory.
- . means the directory you are in right now.
- .. means the directory one level up.

A path that starts at the root, such as /home/student/notes, is called an **absolute** path. A path that starts from where you already are, such as notes or ../shared, is called a **relative** path. Both are useful, and you will move between them without thinking about it before long.

5. Getting around

Three commands do all the navigating.

```
pwd          # print the directory you are in  
ls           # list what is here  
cd Documents # change into the Documents directory  
cd ..        # go up one level  
cd ~         # jump home from anywhere
```

ls becomes far more useful with options.

```
ls -l      # long form: one item per line, with details  
ls -a      # show hidden files, whose names begin with a dot  
ls -lh     # long form with human-readable sizes
```

Notice the pattern. The words after a command that begin with a dash are options that change how it behaves, and you can usually combine them, which is why -lh is the same as -l -h.

6. Creating and removing things

```
mkdir projects      # make a new directory
cd projects
touch todo.txt      # create an empty file
cp todo.txt done.txt # copy a file
mv done.txt old.txt  # move or rename a file
rm old.txt           # delete a file
rmdir projects       # delete an empty directory
```

`mv` both renames and moves, depending on where you point it. If the destination is a new name in the same folder, it renames. If the destination is another folder, it moves.

Careful with `rm`. There is no recycle bin on the command line. A file removed with `rm` is gone for good. Read the line once before you press Enter, and be especially slow with anything that includes a wildcard.

7. Looking at what is inside a file

```
cat todo.txt        # print the whole file to the screen
less todo.txt        # open it in a scrollable pager; press q to quit
head todo.txt        # show only the first lines
tail todo.txt        # show only the last lines
```

`cat` is best for short files, since it dumps everything at once. For anything long, reach for `less`, and remember that `q` quits it.

8. Editing text with vim

The editor we will use most in class is `vim`. It is fast and it is everywhere, but it works differently from most programs, so the first few minutes can feel strange. One idea explains almost all of it: **modes**.

Open a file the usual way.

```
vim notes.txt
```

Vim starts in **Normal mode**, where the keys you press are commands rather than text. This is why typing at first seems to do nothing useful. To enter text, switch to **Insert mode** by pressing `i`. Vim now behaves like an ordinary editor and you can type normally. When you are finished typing, press `Esc` to return to Normal mode.

From Normal mode, commands that begin with a colon save the file and quit.

```
:w      # save (write) the file
:q       # quit
:wq      # save and quit in one step
:q!      # quit without saving
```

If you ever feel stuck. Press **Esc** to make sure you are in Normal mode, then type **:q!** and press Enter. That leaves vim without saving, no matter what state you were in. Almost everyone needs this at least once, so it is worth remembering early.

9. Editing text with nano

If you would rather start with something gentler than vim, **nano** is a friendlier editor to fall back on.

```
nano todo.txt
```

Type normally to add text. The bar along the bottom lists the commands, where the caret symbol means the **Ctrl** key. To save, press **Ctrl+O** and then Enter. To leave, press **Ctrl+X**. That is enough to be productive; the rest can wait.

10. Getting help without memorizing

No one keeps every command in their head. They look things up, and so should you.

```
man ls      # the full manual; press q to quit
ls --help   # a shorter summary printed right in the terminal
apropos copy # search for commands related to a word
```

Many systems also include **tldr**, which shows a few common examples instead of the whole manual. When it is available, it is often the quickest way to remember how a command is used.

11. Redirection and pipes

This is where the command line starts to feel powerful. You can send output into a file, or into another command.

```
echo "first line" > log.txt      # write output into a file, replacing it
echo "second line" >> log.txt    # add to the end instead
cat log.txt | wc -l             # count the lines in the file
ls /usr/bin | grep python       # keep only lines mentioning python
```

A single **>** replaces a file with the output. A double **>>** appends to it. The vertical bar, called a **pipe**, feeds the output of the command on its left into the command on its right. Chaining small tools together with pipes is the central idea of the whole system.

12. Reading permissions

Run **ls -l** and each line begins with a small block of letters like **-rw-r--r--**. That block describes who is allowed to do what.

- The first character is the type: a dash for a file, or **d** for a directory.
- The next three letters are what the **owner** may do: read, write, execute.

- The next three are for the owner's **group**.
- The final three are for **everyone else**.

You can change these with `chmod`. For example, making a script runnable looks like the line below. You will not need this often at first, so it is enough to recognize what the letters mean when you see them.

```
chmod +x myscript.sh    # allow the file to be run as a program
```

13. Installing software

Each distribution has one official **package manager** for adding and updating programs. The command differs by system, but the idea is the same everywhere. Use it rather than downloading programs by hand, because it keeps everything tracked and up to date.

Table 1: Package manager commands by distribution.

Distribution	Install a program	Update everything
XUbuntu (your VM)	<code>sudo apt install NAME</code>	<code>sudo apt update</code>
Arch	<code>sudo pacman -S NAME</code>	<code>sudo pacman -Syu</code>
Void	<code>sudo xbps-install NAME</code>	<code>sudo xbps-install -Su</code>

About sudo. The word `sudo` in front of a command means run this with administrator power. It is required for installing software because that changes the whole system, not just your own files. Use it only when a command genuinely needs it.

14. Habits that save you

A handful of small habits make the terminal far more comfortable.

- **Tab completion:** type the first few letters of a file or command and press **Tab**. The shell finishes the name for you, which also catches typos.
- **Command history:** press the **Up arrow** to bring back previous commands so you can rerun or edit them.
- **Stop a command:** press **Ctrl+C** to cancel something that is running or that you started by mistake.
- **Clear the clutter:** type `clear` to wipe the screen and start fresh.

15. Practice on your own

Try this short sequence without looking back at the guide. If you get stuck, the answer is somewhere above.

1. Move to your home directory and confirm where you are.
2. Create a directory called `practice` and move into it.

3. Create a file called `list.txt` and write three lines into it.
4. Print the file, then count how many lines it has.
5. Make a copy of the file, rename the copy, then delete it.
6. Open the original in `vim` (or `nano`), add one more line, and save.

16. Quick reference

Table 2: Quick reference of core commands.

Command	What it does
<code>pwd</code>	Print the directory you are in
<code>ls</code>	List the contents of a directory
<code>cd</code>	Change directory
<code>mkdir</code>	Make a new directory
<code>touch</code>	Create an empty file
<code>cp</code>	Copy a file
<code>mv</code>	Move or rename a file
<code>rm</code>	Delete a file (no undo)
<code>cat</code>	Print a file to the screen
<code>less</code>	View a file one screen at a time
<code>vim</code>	Edit a file (the one we use most)
<code>nano</code>	Edit a file (simpler alternative)
<code>man</code>	Read the manual for a command
<code>grep</code>	Keep only lines that match a word
<code>clear</code>	Clear the screen

You now have enough to explore comfortably. The best way to learn the rest is to keep a terminal open and reach for it whenever you are curious.